

**Five Year Integrated M.Sc. in Computer Science
(Artificial Intelligence & Data Science)**

**Fifth Semester
Laboratory Record**

24-813-0505: Design and Analysis of Algorithms

*Submitted in partial fulfillment
of the requirements for the award of a degree in
Five Year Integrated M.Sc. in Computer Science
(Artificial Intelligence & Data Science) of
Cochin University of Science and Technology (CUSAT), Kochi*



Submitted by:

Ahmed Sultan P
(Reg No : 24100811)

**DEPARTMENT OF COMPUTER SCIENCE
COCHIN UNIVERSITY OF SCIENCE AND TECHNOLOGY (CUSAT)
KOCHI - 682022
November 2026**

DEPARTMENT OF COMPUTER SCIENCE
COCHIN UNIVERSITY OF SCIENCE AND TECHNOLOGY (CUSAT)
KOCHI, KERALA-682022



*This is to certify that the software laboratory record for **24-813-0505: Design and Analysis of Algorithms** is a record of work carried out by **Ahmed Sultan P (24100811)** in partial fulfillment of the requirements for the award of degree in **Five Year Integrated M.Sc. in Computer Science (Artificial Intelligence & Data Science)** of Cochin University of Science and Technology (CUSAT), Kochi. The lab record has been approved as it satisfies the academic requirements concerning the Fifth-semester laboratory prescribed for the Five Year Integrated M.Sc. in Computer Science degree.*

Faculty Member In-Charge :

Dr. Shailesh Sivan
Assistant Professor
Department of Computer Science
CUSAT

Dr. Bijoy A. Jose
Professor and Head
Department of Computer Science
CUSAT

INDEX

Exp No.	Experiment / Program Title	Page No.
1	Quick Sort	3
2	Merge Sort	7
3	Heap Sort	11
4	Insertion Sort	15
5	Binary Tree Construction & Level Order Traversal	19
6	Binary Search Tree (BST)	23
7	In-order Traversal	27
8	Pre-order Traversal	31
9	Post-order Traversal	35
10	Breadth-First Search (BFS)	39
11	Depth-First Search (DFS)	43
12	Prim's Algorithm	47
13	Kruskal's Algorithm	51
14	Dijkstra's Algorithm	55
15	Bellman-Ford Algorithm	59
16	Floyd-Warshall Algorithm	63

Experiment 1

Quick Sort

QUESTION

Write a C++ program to sort a given sequence of elements using the Quick Sort algorithm.

AIM

To implement the Quick Sort algorithm and verify its working on an array of integers.

THEORY

Quick Sort is a highly efficient, divide-and-conquer sorting algorithm. It selects a 'pivot' element and partitions the other elements into two sub-arrays (less than and greater than the pivot). The sub-arrays are then sorted recursively. It is an in-place sort.

ALGORITHM

1. Select the highest index element as the pivot.
2. Partition the array such that elements smaller than the pivot are on the left and larger are on the right.
3. Recursively apply Quick Sort to the left and right sub-arrays.

COMPLEXITY ANALYSIS

Time: Best/Avg $O(N \log N)$, Worst $O(N^2)$

Space: Avg $O(\log N)$, Worst $O(N)$ (recursion stack)

Output

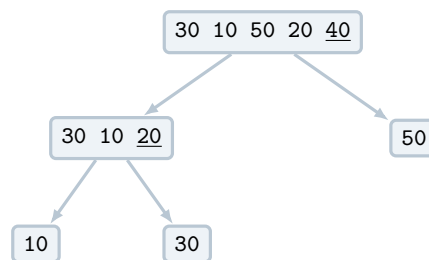
```
Enter the number of elements: 5
Enter the elements: 30 10 50 20 40
Sorted array using Quick Sort: 10 20 30 40 50
```

Result

Each call takes the last element of its segment as the pivot, and **partition** returns the index **pi** at which that pivot finally sits.

Call	Segment (pivot in bold)	Whole array afterwards (pivot underlined)
helper(0,4)	30 10 50 20 40	30 10 20 <u>40</u> 50 pi = 3
helper(0,2)	30 10 20	10 <u>20</u> 30 40 50 pi = 1
helper(0,0)	10	base case, returns immediately
helper(2,2)	30	base case, returns immediately
helper(4,4)	50	base case, returns immediately

Recursion tree (the underlined element is the pivot fixed by that call)



Reading the fixed pivots and leaves from left to right gives 10 20 30 40 50.

Program

```
1  #include <iostream>
2  #include <vector>
3  using namespace std;
4  int partition(vector<int>& arr, int low, int high) {
5      int pivot = arr[high];
6      int i = (low - 1);
7      for (int j = low; j <= high - 1; j++) {
8          if (arr[j] <= pivot) {
9              i++;
10             swap(arr[i], arr[j]);
11         }
12     }
13     swap(arr[i + 1], arr[high]);
14     return (i + 1);
15 }
16 void quick_sort_helper(vector<int>& arr, int low, int high) {
17     if (low < high) {
18         int pi = partition(arr, low, high);
19         quick_sort_helper(arr, low, pi - 1);
20         quick_sort_helper(arr, pi + 1, high);
21     }
22 }
23 void quick_sort(vector<int>& arr) {
24     if (!arr.empty()) {
25         quick_sort_helper(arr, 0, arr.size() - 1);
26     }
27 }
28 int main() {
29     int size;
30     cout << "Enter the number of elements: ";
31     cin >> size;
32     vector<int> arr(size);
33     cout << "Enter the elements: ";
34     for (int i = 0; i < size; i++) cin >> arr[i];
35     quick_sort(arr);
36     cout << "Sorted array using Quick Sort:";
37     for (int i = 0; i < size; i++) cout << " " << arr[i];
38     cout << endl;
39     return 0;
40 }
```


Experiment 2

Merge Sort

QUESTION

Write a C++ program to sort an array of integers using the Merge Sort algorithm.

AIM

To implement the Merge Sort algorithm and demonstrate its divide-and-conquer approach.

THEORY

Merge Sort is a stable, divide-and-conquer sorting algorithm. It divides the input array into two halves, recursively sorts the two halves, and then merges the sorted halves. It guarantees $O(N \log N)$ time complexity but requires extra space.

ALGORITHM

1. Find the middle point to divide the array into two halves.
2. Recursively call Merge Sort for the first half and the second half.
3. Merge the two sorted halves into a single sorted array using temporary arrays.

COMPLEXITY ANALYSIS

Time: Best/Avg/Worst $O(N \log N)$

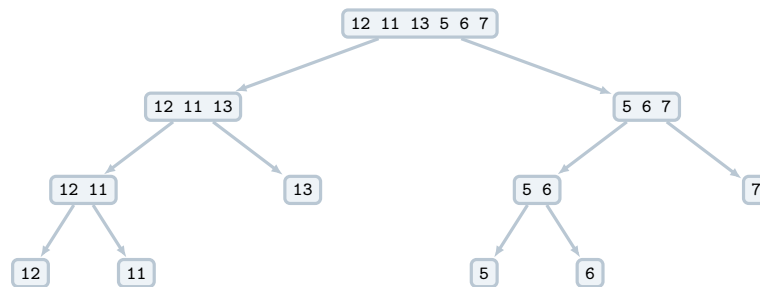
Space: $O(N)$

Output

```
Enter the number of elements: 6
Enter the elements: 12 11 13 5 6 7
Sorted array using Merge Sort: 5 6 7 11 12 13
```

Result

The array is split until every part holds a single element, then the parts are merged back in order. Splits run top-down, merges bottom-up.



Merge steps, in the order they actually execute

Call	Left run	Right run	Merged result
merge(0,0,1)	12	11	11 12
merge(0,1,2)	11 12	13	11 12 13
merge(3,3,4)	5	6	5 6
merge(3,4,5)	5 6	7	5 6 7
merge(0,2,5)	11 12 13	5 6 7	5 6 7 11 12 13

Program

```
1  #include <iostream>
2  #include <vector>
3  #include <climits>
4  using namespace std;
5  void merge(vector<int>& arr, int left, int mid, int right) {
6      int n1 = mid - left + 1;
7      int n2 = right - mid;
8      vector<int> L(n1 + 1), R(n2 + 1);
9      for (int i = 0; i < n1; i++) L[i] = arr[left + i];
10     for (int j = 0; j < n2; j++) R[j] = arr[mid + 1 + j];
11     L[n1] = INT_MAX;
12     R[n2] = INT_MAX;
13     int i = 0, j = 0;
14     for (int k = left; k <= right; k++) {
15         if (L[i] <= R[j]) {
16             arr[k] = L[i];
17             i++;
18         } else {
19             arr[k] = R[j];
20             j++;
21         }
22     }
23 }
24 void merge_sort_helper(vector<int>& arr, int left, int right) {
25     if (left < right) {
26         int mid = left + (right - left) / 2;
27         merge_sort_helper(arr, left, mid);
28         merge_sort_helper(arr, mid + 1, right);
29         merge(arr, left, mid, right);
30     }
31 }
32 void merge_sort(vector<int>& arr) {
33     if (!arr.empty()) merge_sort_helper(arr, 0, arr.size() - 1);
34 }
35 int main() {
36     int size;
37     cout << "Enter the number of elements: ";
38     cin >> size;
39     vector<int> arr(size);
40     cout << "Enter the elements: ";
41     for (int i = 0; i < size; i++) cin >> arr[i];
42     merge_sort(arr);
43     cout << "Sorted array using Merge Sort:";
44     for (int i = 0; i < size; i++) cout << " " << arr[i];
45     cout << endl;
46     return 0;
47 }
```


Experiment 3

Heap Sort

QUESTION

Write a C++ program to sort an array using the Heap Sort algorithm.

AIM

To implement the Heap Sort algorithm using a Max-Heap data structure.

THEORY

Heap Sort is a comparison-based technique based on a Binary Heap. It builds a Max-Heap from the data, swaps the largest element (root) with the last item, reduces the heap size, and heapifies the new root. This repeats until sorted.

ALGORITHM

1. Build a Max-Heap from the input array.
2. Swap the root with the last element and reduce the heap size by 1.
3. Call `heapify` on the root to restore the Max-Heap property.
4. Repeat steps 2-3 until all elements are sorted.

COMPLEXITY ANALYSIS

Time: Best/Avg/Worst $O(N \log N)$

Space: $O(\log N)$ (recursion stack)

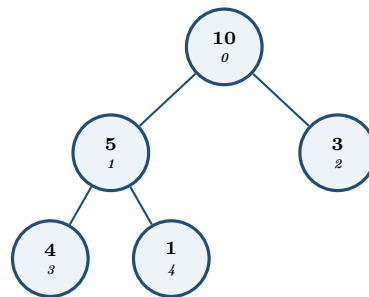
Output

```
Enter the number of elements: 5
Enter the elements: 4 10 3 5 1
Sorted array using Heap Sort: 1 3 4 5 10
```

Result

Stage 1 — build the max-heap. The array is read as a complete binary tree in which index i has children $2i + 1$ and $2i + 2$. Building calls **heapify** from index $n/2 - 1 = 1$ down to 0.

Call	Effect and resulting array
heapify(5,1)	10 already dominates its children 5 and 1, so no swap: 4 10 3 5 1
heapify(5,0)	4 swaps with 10, then sinks and swaps with 5: 10 5 3 4 1



The max-heap 10 5 3 4 1; the small italic number is the array index.

Stage 2 — extract the root repeatedly. The largest element is swapped to the end, the heap shrinks by one, and the new root sinks back into place.

Swap	New size	Sinking of the new root	Array
root 10 $\leftrightarrow$ a[4]=1	4	1 swaps with 5, then with 4	5 4 3 1 10
root 5 $\leftrightarrow$ a[3]=1	3	1 swaps with 4	4 1 3 5 10
root 4 $\leftrightarrow$ a[2]=3	2	already a heap, no swap	3 1 4 5 10
root 3 $\leftrightarrow$ a[1]=1	1	single element, nothing to do	1 3 4 5 10

Program

```
1  #include <iostream>
2  #include <vector>
3  using namespace std;
4  void heapify(vector<int>& arr, int n, int i) {
5      int largest = i;
6      int left = 2 * i + 1;
7      int right = 2 * i + 2;
8      if (left < n && arr[left] > arr[largest])
9          largest = left;
10     if (right < n && arr[right] > arr[largest])
11         largest = right;
12     if (largest != i) {
13         swap(arr[i], arr[largest]);
14         heapify(arr, n, largest);
15     }
16 }
17 void heap_sort(vector<int>& arr) {
18     int n = arr.size();
19     for (int i = n / 2 - 1; i >= 0; i--)
20         heapify(arr, n, i);
21     for (int i = n - 1; i > 0; i--) {
22         swap(arr[0], arr[i]);
23         heapify(arr, i, 0);
24     }
25 }
26 int main() {
27     int size;
28     cout << "Enter the number of elements: ";
29     cin >> size;
30     vector<int> arr(size);
31     cout << "Enter the elements: ";
32     for (int i = 0; i < size; i++) cin >> arr[i];
33     heap_sort(arr);
34     cout << "Sorted array using Heap Sort:";
35     for (int i = 0; i < size; i++) cout << " " << arr[i];
36     cout << endl;
37     return 0;
38 }
```


Experiment 4

Insertion Sort

QUESTION

Write a C++ program to sort elements using the Insertion Sort algorithm.

AIM

To sort an array of integers by repeatedly placing each element in its proper position in a sorted subsection.

THEORY

Insertion sort builds the final sorted array one item at a time. It iterates through the input elements, growing a sorted list by comparing the current element to the already sorted ones and inserting it into the correct position.

ALGORITHM

1. Iterate from the second element to the last element of the array.
2. Compare the current element (**key**) with the sorted elements before it.
3. Shift all elements greater than the **key** one position to the right.
4. Insert the **key** into its correct position.

COMPLEXITY ANALYSIS

Time: Best $O(N)$, Avg/Worst $O(N^2)$

Space: $O(1)$

Output

```
Enter the number of elements: 5
Enter the elements: 12 11 13 5 6
Sorted array using Insertion Sort: 5 6 11 12 13
```

Result

At the start of pass i the elements to the left of position i are already sorted. `key = arr[i]` is lifted out, every larger element to its left shifts one place right, and the key drops into the gap.

Pass i	key	Elements shifted	Array after the insertion (key in brackets)
1	11	1	[11] 12 13 5 6
2	13	0	11 12 [13] 5 6 already in place
3	5	3	[5] 11 12 13 6
4	6	3	5 [6] 11 12 13

Pass 2 does no shifting at all, which is why an already-sorted input costs only $O(N)$; passes 3 and 4 each shift three elements, which is the $O(N^2)$ behaviour.

Program

```
1  #include <iostream>
2  #include <vector>
3  using namespace std;
4  void insertion_sort(vector<int>& arr) {
5      int n = arr.size();
6      for (int i = 1; i < n; i++) {
7          int key = arr[i];
8          int j = i - 1;
9          while (j >= 0 && arr[j] > key) {
10             arr[j + 1] = arr[j];
11             j--;
12         }
13         arr[j + 1] = key;
14     }
15 }
16 int main() {
17     int size;
18     cout << "Enter the number of elements: ";
19     cin >> size;
20     vector<int> arr(size);
21     cout << "Enter the elements: ";
22     for (int i = 0; i < size; i++) cin >> arr[i];
23     insertion_sort(arr);
24     cout << "Sorted array using Insertion Sort:";
25     for (int i = 0; i < size; i++) cout << " " << arr[i];
26     cout << endl;
27     return 0;
28 }
```


Experiment 5

Binary Tree Construction & Level Order Traversal

QUESTION

Write a C++ program to dynamically build a Binary Tree and perform level-order traversal.

AIM

To implement a Binary Tree using nodes and display its elements using breadth-first traversal.

THEORY

A Binary Tree is a structure in which each node has at most two children. Level-order traversal visits nodes level by level from top to bottom and left to right, typically implemented using a Queue.

ALGORITHM

1. Initialize a queue with the root node.
2. While the queue is not empty, dequeue the front node and print its data.
3. If the left child exists, enqueue it. If the right child exists, enqueue it.

COMPLEXITY ANALYSIS

Time: $O(N)$

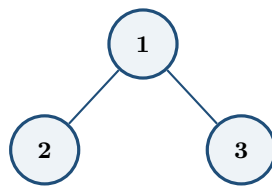
Space: $O(N)$

Output

```
Enter root value (-1 for no node): 1
Enter left child of 1 (-1 for none): 2
Enter right child of 1 (-1 for none): 3
Enter left child of 2 (-1 for none): -1
Enter right child of 2 (-1 for none): -1
Enter left child of 3 (-1 for none): -1
Enter right child of 3 (-1 for none): -1
Level Order Traversal: 1 2 3
```

Result

The prompts above build this tree; every -1 closes off a child pointer.



Queue during level-order traversal

Step	Dequeued	Printed	Queue after the step
0	—	—	[1] root enqueued before the loop
1	1	1	[2, 3] both children exist, so both enqueue
2	2	2	[3] 2 is a leaf, nothing enqueued
3	3	3	[] 3 is a leaf, queue empties

Because the queue is FIFO, nodes leave it in the order they were discovered, which is exactly level by level: 1 2 3.

Program

```

1  #include <iostream>
2  #include <queue>
3  using namespace std;
4  struct Node {
5      int data;
6      Node* left; Node* right;
7      Node(int val) : data(val), left(nullptr), right(nullptr) {}
8  };
9  Node* build_binary_tree() {
10     int val;
11     cout << "Enter root value (-1 for no node): ";
12     cin >> val;
13     if (val == -1) return nullptr;
14     Node* root = new Node(val);
15     queue<Node*> q;
16     q.push(root);
17     while (!q.empty()) {
18         Node* curr = q.front(); q.pop();
19         int leftVal, rightVal;
20         cout << "Enter left child of " << curr->data << " (-1 for none): ";
21         cin >> leftVal;
22         if (leftVal != -1) {
23             curr->left = new Node(leftVal);
24             q.push(curr->left);
25         }
26         cout << "Enter right child of " << curr->data << " (-1 for none): ";
27         cin >> rightVal;
28         if (rightVal != -1) {
29             curr->right = new Node(rightVal);
30             q.push(curr->right);
31         }
32     }
33     return root;
34 }
35 void level_order(Node* root) {
36     if (!root) return;
37     queue<Node*> q; q.push(root);
38     cout << "Level Order Traversal:";
39     while (!q.empty()) {
40         Node* curr = q.front(); q.pop();
41         cout << " " << curr->data;
42         if (curr->left) q.push(curr->left);
43         if (curr->right) q.push(curr->right);
44     }
45     cout << endl;
46 }
47 int main() {
48     Node* root = build_binary_tree();
49     level_order(root);
50     return 0;
51 }

```


Experiment 6

Binary Search Tree (BST)

QUESTION

Write a C++ program to insert elements into a Binary Search Tree and print them in In-order traversal.

AIM

To implement the insertion logic of a BST and verify its sorting property via In-order traversal.

THEORY

A BST is a binary tree where the left subtree contains keys lesser than the root, and the right subtree contains keys greater than the root. In-order traversal yields the elements in ascending order.

ALGORITHM

1. To insert, check if the root is null. If yes, create a new node.
2. If $\text{value} < \text{root}$, recursively insert into the left subtree.
3. If $\text{value} > \text{root}$, recursively insert into the right subtree.

COMPLEXITY ANALYSIS

Time: Per insertion: Avg $O(\log N)$, Worst $O(N)$. Building the BST from N keys: Avg $O(N \log N)$, Worst $O(N^2)$

Space: $O(N)$

Output

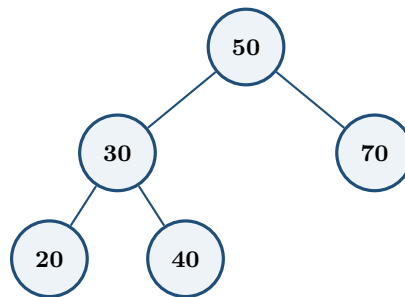
```
Enter the number of elements: 5
Enter the elements: 50 30 70 20 40
In-order Traversal of BST: 20 30 40 50 70
```

Result

Insertion path of each key

Key	Comparisons made, and where it lands
-----	--------------------------------------

50	tree is empty, so 50 becomes the root
30	$30 < 50 \Rightarrow$ left of 50
70	$70 > 50 \Rightarrow$ right of 50
20	$20 < 50$, then $20 < 30 \Rightarrow$ left of 30
40	$40 < 50$, then $40 > 30 \Rightarrow$ right of 30



Every key in the left subtree of a node is smaller than it and every key in the right subtree is larger, so visiting left $\rightarrow$ node $\rightarrow$ right reads the keys off in ascending order: 20 30 40 50 70. The in-order traversal is therefore a direct check that the BST property holds.

Program

```
1  #include <iostream>
2  using namespace std;
3  struct Node {
4      int data;
5      Node* left; Node* right;
6      Node(int val) : data(val), left(nullptr), right(nullptr) {}
7  };
8  Node* insert(Node* root, int val) {
9      if (!root) return new Node(val);
10     if (val < root->data)
11         root->left = insert(root->left, val);
12     else if (val > root->data)
13         root->right = insert(root->right, val);
14     return root;
15 }
16 void inorder(Node* root) {
17     if (!root) return;
18     inorder(root->left);
19     cout << " " << root->data;
20     inorder(root->right);
21 }
22 int main() {
23     int size, val;
24     cout << "Enter the number of elements: ";
25     cin >> size;
26     Node* root = nullptr;
27     cout << "Enter the elements: ";
28     for (int i = 0; i < size; i++) {
29         cin >> val;
30         root = insert(root, val);
31     }
32     cout << "In-order Traversal of BST:";
33     inorder(root);
34     cout << endl;
35     return 0;
36 }
```


Experiment 7

In-order Traversal

QUESTION

Write a C++ program to build a binary tree and perform In-order traversal.

AIM

To implement the depth-first In-order traversal technique for a Binary Tree.

THEORY

In-order traversal is a depth-first search strategy where the nodes are visited in the order: Left Subtree → Root Node → Right Subtree.

ALGORITHM

1. Build a binary tree level by level using a queue.
2. Traverse the left subtree recursively.
3. Visit (print) the current root node.
4. Traverse the right subtree recursively.

COMPLEXITY ANALYSIS

Time: $O(N)$

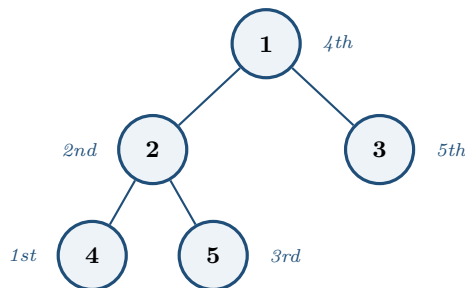
Space: $O(N)$

Output

```
Enter root value (-1 for no node): 1
Enter left child of 1 (-1 for none): 2
Enter right child of 1 (-1 for none): 3
Enter left child of 2 (-1 for none): 4
Enter right child of 2 (-1 for none): 5
Enter left child of 3 (-1 for none): -1
Enter right child of 3 (-1 for none): -1
Enter left child of 4 (-1 for none): -1
Enter right child of 4 (-1 for none): -1
Enter left child of 5 (-1 for none): -1
Enter right child of 5 (-1 for none): -1
In-Order Traversal: 4 2 5 1 3
```

Result

The prompts above build the tree below. The circled number is the node value; the italic number beside it is the position at which in-order traversal prints it.



Order of recursive calls

Call	What happens
<code>in_order(1)</code>	descends left before printing anything
<code>in_order(2)</code>	descends left again
<code>in_order(4)</code>	left child is null, so 4 is printed , right child is null, returns
back in (2)	2 is printed , then the right subtree is entered
<code>in_order(5)</code>	leaf, so 5 is printed
back in (1)	1 is printed , then the right subtree is entered
<code>in_order(3)</code>	leaf, so 3 is printed

Left subtree → root → right subtree gives 4 2 5 1 3.

Program

```
1 #include <iostream>
2 #include <queue>
3 using namespace std;
4 struct Node {
5     int data;
6     Node* left; Node* right;
7     Node(int val) : data(val), left(nullptr), right(nullptr) {}
8 };
9 Node* build_tree() {
10     int val;
11     cout << "Enter root value (-1 for no node): ";
12     cin >> val;
13     if (val == -1) return nullptr;
14     Node* root = new Node(val);
15     queue<Node*> q;
16     q.push(root);
17     while (!q.empty()) {
18         Node* curr = q.front(); q.pop();
19         int leftVal, rightVal;
20         cout << "Enter left child of " << curr->data << " (-1 for none): ";
21         cin >> leftVal;
22         if (leftVal != -1) { curr->left = new Node(leftVal); q.push(curr->left); }
23         cout << "Enter right child of " << curr->data << " (-1 for none): ";
24         cin >> rightVal;
25         if (rightVal != -1) { curr->right = new Node(rightVal); q.push(curr->right); }
26     }
27     return root;
28 }
29 void in_order(Node* root) {
30     if (!root) return;
31     in_order(root->left);
32     cout << " " << root->data;
33     in_order(root->right);
34 }
35 int main() {
36     Node* root = build_tree();
37     cout << "In-Order Traversal:";
38     in_order(root);
39     cout << endl;
40     return 0;
41 }
```


Experiment 8

Pre-order Traversal

QUESTION

Write a C++ program to construct a binary tree and perform Pre-order traversal.

AIM

To explore and implement the Pre-order traversal of a binary data structure.

THEORY

Pre-order traversal visits nodes in the sequence: Root Node $\rightarrow$ Left Subtree $\rightarrow$ Right Subtree. This is often used to copy a tree or evaluate expression trees.

ALGORITHM

1. Build a binary tree level by level.
2. Visit (print) the data of the current root.
3. Recursively traverse the left child.
4. Recursively traverse the right child.

COMPLEXITY ANALYSIS

Time: $O(N)$

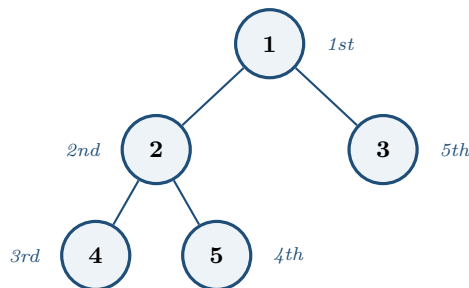
Space: $O(N)$

Output

```
Enter root value (-1 for no node): 1
Enter left child of 1 (-1 for none): 2
Enter right child of 1 (-1 for none): 3
Enter left child of 2 (-1 for none): 4
Enter right child of 2 (-1 for none): 5
Enter left child of 3 (-1 for none): -1
Enter right child of 3 (-1 for none): -1
Enter left child of 4 (-1 for none): -1
Enter right child of 4 (-1 for none): -1
Enter left child of 5 (-1 for none): -1
Enter right child of 5 (-1 for none): -1
Pre-Order Traversal: 1 2 4 5 3
```

Result

The same tree as before. The italic number beside each node is the position at which pre-order traversal prints it — the root is printed on arrival, before either child.



Order of recursive calls

Call	What happens
------	--------------

pre_order(1)	1 is printed first , then the left subtree is entered
--------------	--

pre_order(2)	2 is printed , then the left subtree is entered
--------------	--

pre_order(4)	leaf, 4 is printed , returns
--------------	-------------------------------------

pre_order(5)	leaf, 5 is printed , returns to 1
--------------	--

pre_order(3)	leaf, 3 is printed
--------------	---------------------------

Root → left subtree → right subtree gives 1 2 4 5 3. Printing the root first is what makes pre-order suitable for copying a tree: each node is emitted before the children that must be attached to it.

Program

```
1  #include <iostream>
2  #include <queue>
3  using namespace std;
4  struct Node {
5      int data;
6      Node* left; Node* right;
7      Node(int val) : data(val), left(nullptr), right(nullptr) {}
8  };
9  Node* build_tree() {
10     int val;
11     cout << "Enter root value (-1 for no node): ";
12     cin >> val;
13     if (val == -1) return nullptr;
14     Node* root = new Node(val);
15     queue<Node*> q;
16     q.push(root);
17     while (!q.empty()) {
18         Node* curr = q.front(); q.pop();
19         int leftVal, rightVal;
20         cout << "Enter left child of " << curr->data << " (-1 for none): ";
21         cin >> leftVal;
22         if (leftVal != -1) { curr->left = new Node(leftVal); q.push(curr->left); }
23         cout << "Enter right child of " << curr->data << " (-1 for none): ";
24         cin >> rightVal;
25         if (rightVal != -1) { curr->right = new Node(rightVal); q.push(curr->right); }
26     }
27     return root;
28 }
29 void pre_order(Node* root) {
30     if (!root) return;
31     cout << " " << root->data;
32     pre_order(root->left);
33     pre_order(root->right);
34 }
35 int main() {
36     Node* root = build_tree();
37     cout << "Pre-Order Traversal: ";
38     pre_order(root);
39     cout << endl;
40     return 0;
41 }
```


Experiment 9

Post-order Traversal

QUESTION

Write a C++ program to build a binary tree and demonstrate Post-order traversal.

AIM

To implement the Post-order traversal logic on a Binary Tree structure.

THEORY

Post-order traversal visits nodes in the sequence: Left Subtree $\rightarrow$ Right Subtree $\rightarrow$ Root Node. It is commonly used when deleting trees.

ALGORITHM

1. Construct a binary tree level by level.
2. Recursively traverse the left subtree.
3. Recursively traverse the right subtree.
4. Visit (print) the data of the current root.

COMPLEXITY ANALYSIS

Time: $O(N)$

Space: $O(N)$

Output

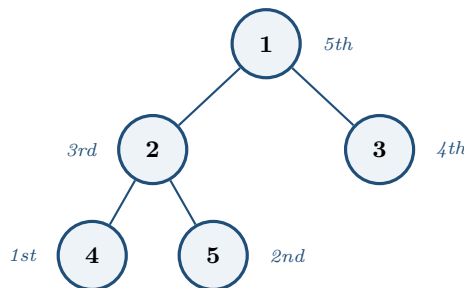
```

Enter root value (-1 for no node): 1
Enter left child of 1 (-1 for none): 2
Enter right child of 1 (-1 for none): 3
Enter left child of 2 (-1 for none): 4
Enter right child of 2 (-1 for none): 5
Enter left child of 3 (-1 for none): -1
Enter right child of 3 (-1 for none): -1
Enter left child of 4 (-1 for none): -1
Enter right child of 4 (-1 for none): -1
Enter left child of 5 (-1 for none): -1
Enter right child of 5 (-1 for none): -1
Post-Order Traversal: 4 5 2 3 1

```

Result

The same tree again. The italic number beside each node is the position at which post-order traversal prints it — a node is printed only after both of its subtrees are done.



Order of recursive calls

Call	What happens
post_order(1)	both subtrees must finish before 1 may be printed
post_order(2)	its own subtrees must finish first as well
post_order(4)	leaf, 4 is printed
post_order(5)	leaf, 5 is printed
back in (2)	both children done, so 2 is printed
post_order(3)	leaf, 3 is printed
back in (1)	both subtrees done, so 1 is printed last

Left subtree → right subtree → root gives 4 5 2 3 1. Because every child is emitted before its parent, this is the safe order in which to delete a tree: no node is freed while another node still points to it.

Program

```
1  #include <iostream>
2  #include <queue>
3  using namespace std;
4  struct Node {
5      int data;
6      Node* left; Node* right;
7      Node(int val) : data(val), left(nullptr), right(nullptr) {}
8  };
9  Node* build_tree() {
10     int val;
11     cout << "Enter root value (-1 for no node): ";
12     cin >> val;
13     if (val == -1) return nullptr;
14     Node* root = new Node(val);
15     queue<Node*> q;
16     q.push(root);
17     while (!q.empty()) {
18         Node* curr = q.front(); q.pop();
19         int leftVal, rightVal;
20         cout << "Enter left child of " << curr->data << " (-1 for none): ";
21         cin >> leftVal;
22         if (leftVal != -1) { curr->left = new Node(leftVal); q.push(curr->left); }
23         cout << "Enter right child of " << curr->data << " (-1 for none): ";
24         cin >> rightVal;
25         if (rightVal != -1) { curr->right = new Node(rightVal); q.push(curr->right); }
26     }
27     return root;
28 }
29 void post_order(Node* root) {
30     if (!root) return;
31     post_order(root->left);
32     post_order(root->right);
33     cout << " " << root->data;
34 }
35 int main() {
36     Node* root = build_tree();
37     cout << "Post-Order Traversal:";
38     post_order(root);
39     cout << endl;
40     return 0;
41 }
```


Experiment 10

Breadth-First Search (BFS)

QUESTION

Write a C++ program to perform Breadth-First Search (BFS) on an adjacency list graph.

AIM

To implement the BFS algorithm to systematically visit graph vertices level-by-level.

THEORY

BFS explores all neighbor nodes at the present depth before moving on to the next depth level. It uses a queue and a visited array to maintain traversal order and avoid cycles.

ALGORITHM

1. Enqueue the starting node and mark it as visited.
2. While queue is not empty, dequeue a node and print it.
3. Iterate adjacent nodes. If unvisited, mark visited and enqueue.

COMPLEXITY ANALYSIS

Time: $O(V + E)$

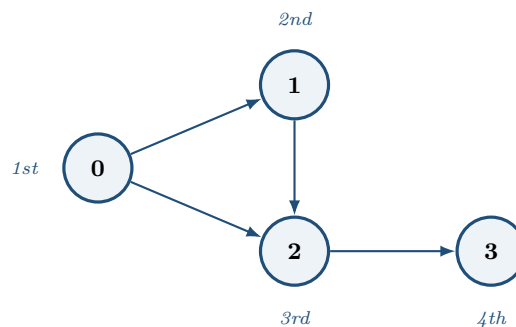
Space: $O(V)$

Output

```
Enter the number of nodes: 4
Enter Neighbour Vertex of Vertex 0 (Enter -1 to stop): 1
Enter Neighbour Vertex of Vertex 0 (Enter -1 to stop): 2
Enter Neighbour Vertex of Vertex 0 (Enter -1 to stop): -1
Enter Neighbour Vertex of Vertex 1 (Enter -1 to stop): 2
Enter Neighbour Vertex of Vertex 1 (Enter -1 to stop): -1
Enter Neighbour Vertex of Vertex 2 (Enter -1 to stop): 3
Enter Neighbour Vertex of Vertex 2 (Enter -1 to stop): -1
Enter Neighbour Vertex of Vertex 3 (Enter -1 to stop): -1
Enter the starting node: 0
0 1 2 3
```

Result

The adjacency list read above describes the directed graph below. The label beside each vertex is the order in which BFS prints it.



Queue evolution

Step	Dequeued	Printed	Queue after the step
0	—	—	[0] start marked visited on entry
1	0	0	[1, 2] 1 and 2 marked visited
2	1	1	[2] neighbour 2 already visited, skipped
3	2	2	[3] 3 marked visited
4	3	3	[] 3 has no neighbours, queue empties

Program

```
1  #include<iostream>
2  #include<vector>
3  #include<queue>
4  using namespace std;
5  void bfs(const vector<vector<int>>& adj, int start, int size) {
6      queue<int> q;
7      vector<bool> is_visited(size, false);
8      q.push(start);
9      is_visited[start]=true;
10     while(!q.empty()) {
11         int curr = q.front();
12         cout<<" "<<curr;
13         q.pop();
14         for (int v : adj[curr]) {
15             if (!is_visited[v]) {
16                 is_visited[v]=true;
17                 q.push(v);
18             }
19         }
20     }
21 }
22 int main() {
23     int size, start, v;
24     cout << "Enter the number of nodes: ";
25     cin >> size;
26     vector<vector<int>> adj(size);
27     for (int i = 0; i < size; i++) {
28         do {
29             cout << "Enter Neighbour Vertex of Vertex " << i << " (Enter -1 to stop): ";
30             cin >> v;
31             if (v == -1) break;
32             adj[i].push_back(v);
33         } while (true);
34     }
35     cout << "Enter the starting node: ";
36     cin >> start;
37     bfs(adj, start, size);
38     cout << endl;
39     return 0;
40 }
```


Experiment 11

Depth-First Search (DFS)

QUESTION

Write a C++ program to traverse a graph using the Depth-First Search (DFS) algorithm.

AIM

To recursively explore a graph structure utilizing the DFS methodology.

THEORY

DFS explores as far as possible along each branch before backtracking. It relies heavily on recursion (which acts as a stack) and a visited array to prevent infinite loops.

ALGORITHM

1. Define a recursive helper function passing the current node.
2. Mark the current node as visited and print its value.
3. For all adjacent unvisited nodes, recursively call the function.

COMPLEXITY ANALYSIS

Time: $O(V + E)$

Space: $O(V)$

Output

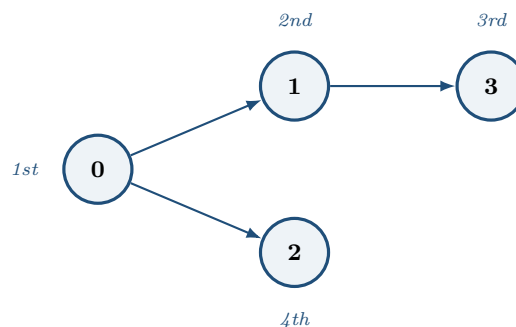
```

Enter the number of nodes: 4
Enter Neighbour Vertex of Vertex 0 (Enter -1 to stop): 1
Enter Neighbour Vertex of Vertex 0 (Enter -1 to stop): 2
Enter Neighbour Vertex of Vertex 0 (Enter -1 to stop): -1
Enter Neighbour Vertex of Vertex 1 (Enter -1 to stop): 3
Enter Neighbour Vertex of Vertex 1 (Enter -1 to stop): -1
Enter Neighbour Vertex of Vertex 2 (Enter -1 to stop): -1
Enter Neighbour Vertex of Vertex 3 (Enter -1 to stop): -1
Enter the starting node: 0
0 1 3 2

```

Result

The adjacency list read above gives the directed graph below. The label beside each vertex is the order in which DFS prints it.



Recursion stack, indented by depth

Call	Depth	What happens
dfs_helper(0)	1	mark 0, print 0 , take first neighbour 1
dfs_helper(1)	2	mark 1, print 1 , take its only neighbour 3
dfs_helper(3)	3	mark 3, print 3 , no neighbours, return
back in (1)	2	neighbour list exhausted, return
back in (0)	1	next neighbour is 2, still unvisited
dfs_helper(2)	2	mark 2, print 2 , no neighbours, return

DFS follows the branch $0 \rightarrow 1 \rightarrow 3$ to its end before backtracking to take 0's second neighbour, giving 0 1 3 2. Compare this with Experiment 10, where the queue-based BFS on a similar graph instead sweeps level by level.

Program

```
1  #include<iostream>
2  #include<vector>
3  using namespace std;
4  void dfs_helper(const vector<vector<int>>& adj, int curr, vector<bool>& is_visited) {
5      is_visited[curr] = true;
6      cout << " " << curr;
7      for (int v : adj[curr]) {
8          if (!is_visited[v]) {
9              dfs_helper(adj, v, is_visited);
10         }
11     }
12 }
13 void dfs(const vector<vector<int>>& adj, int start, int size) {
14     vector<bool> is_visited(size, false);
15     dfs_helper(adj, start, is_visited);
16 }
17 int main() {
18     int size, start, v;
19     cout << "Enter the number of nodes: ";
20     cin >> size;
21     vector<vector<int>> adj(size);
22     for (int i = 0; i < size; i++) {
23         do {
24             cout << "Enter Neighbour Vertex of Vertex " << i << " (Enter -1 to stop): ";
25             cin >> v;
26             if (v == -1) break;
27             adj[i].push_back(v);
28         } while (true);
29     }
30     cout << "Enter the starting node: ";
31     cin >> start;
32     dfs(adj, start, size);
33     cout << endl;
34     return 0;
35 }
```


Experiment 12

Prim's Algorithm

QUESTION

Write a C++ program to find the Minimum Spanning Tree (MST) of a graph using Prim's algorithm.

AIM

To implement Prim's algorithm using a Priority Queue and calculate the total weight.

THEORY

Prim's algorithm builds an MST by starting from an arbitrary vertex and at each step adding the cheapest possible connection from the tree to another unvisited vertex.

ALGORITHM

1. Push the starting node into a Min-Heap (weight 0, parent -1).
2. Extract the minimum edge. If unvisited, mark visited and add to MST.
3. Push all unvisited adjacent nodes and their weights into the Min-Heap.

COMPLEXITY ANALYSIS

Time: $O(E \log V)$

Space: $O(V + E)$

Output

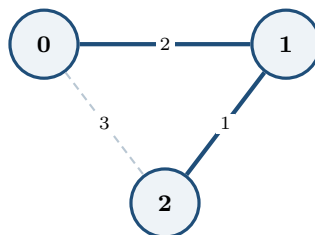
```

Enter the number of vertices: 3
Enter Neighbour Vertex Distance of Vertex 0 (-1,-1 To Stop Reading): 1 2
Enter Neighbour Vertex Distance of Vertex 0 (-1,-1 To Stop Reading): 2 3
Enter Neighbour Vertex Distance of Vertex 0 (-1,-1 To Stop Reading): -1 -1
Enter Neighbour Vertex Distance of Vertex 1 (-1,-1 To Stop Reading): 0 2
Enter Neighbour Vertex Distance of Vertex 1 (-1,-1 To Stop Reading): 2 1
Enter Neighbour Vertex Distance of Vertex 1 (-1,-1 To Stop Reading): -1 -1
Enter Neighbour Vertex Distance of Vertex 2 (-1,-1 To Stop Reading): 0 3
Enter Neighbour Vertex Distance of Vertex 2 (-1,-1 To Stop Reading): 1 1
Enter Neighbour Vertex Distance of Vertex 2 (-1,-1 To Stop Reading): -1 -1
Edges in the Minimum Spanning Tree:
0 - 1 : 2
1 - 2 : 1
Total Weight of MST: 3

```

Result

Each edge was entered from both of its endpoints, so the adjacency list describes this undirected weighted graph. Thick edges are the ones Prim's algorithm keeps.



Solid = in the MST, dashed = considered then discarded.

Min-heap and MST growth. Entries are written (weight: parent→vertex).

Step	Popped	Decision, then the heap contents
—	—	start: {(0: -1→0)}
1	(0, v=0, p=-1)	start vertex, no edge recorded; heap {(2:0→1), (3:0→2)}
2	(2, v=1, p=0)	1 is new ⇒ accept 0–1, weight 2 , total 2; heap {(1:1→2), (3:0→2)}
3	(1, v=2, p=1)	2 is new ⇒ accept 1–2, weight 1 , total 3; heap {(3:0→2)}
4	(3, v=2, p=0)	2 is already in the MST ⇒ discard; heap empty

All 3 vertices were reached, so the graph is connected and the spanning tree is complete: edges 0–1 and 1–2, total weight 3. Note that edge 0–2 of weight 3 sat in the heap the whole time and was rejected at step 4 — the cheaper route to vertex 2 was found first through vertex 1.

Program

```

1  #include <iostream>
2  #include <vector>
3  #include <queue>
4  using namespace std;
5  void run_prims(const vector<vector<pair<int, int>>>& adj, int size) {
6      priority_queue<pair<int, pair<int, int>>, vector<pair<int, pair<int, int>>>, greater<>> pq;
7      vector<bool> in_mst(size, false);
8      pq.push({0, {0, -1}});
9      int mst_weight = 0, visited_count = 0;
10     cout << "Edges in the Minimum Spanning Tree:\n";
11     while (!pq.empty()) {
12         auto [w, edge] = pq.top(); pq.pop();
13         auto [u, parent] = edge;
14         if (in_mst[u]) continue;
15         in_mst[u] = true;
16         visited_count++;
17         if (parent != -1) {
18             cout << parent << " - " << u << " : " << w << endl;
19             mst_weight += w;
20         }
21         for (auto [v, weight] : adj[u]) {
22             if (!in_mst[v]) pq.push({weight, {v, u}});
23         }
24     }
25     if (visited_count != size) {
26         cout << "Graph is disconnected: no spanning tree exists" << endl;
27         return;
28     }
29     cout << "Total Weight of MST: " << mst_weight << endl;
30 }
31 int main() {
32     int v, d, size;
33     cout << "Enter the number of vertices: ";
34     cin >> size;
35     vector<vector<pair<int, int>>> adj(size);
36     for (int i = 0; i < size; i++) {
37         do {
38             cout << "Enter Neighbour Vertex Distance of Vertex " << i << " (-1,-1 To Stop Reading): ";
39             ;
40             cin >> v >> d;
41             if (v == -1 && d == -1) break;
42             adj[i].push_back({v, d});
43         } while (true);
44     }
45     run_prims(adj, size);
46     return 0;
47 }

```


Experiment 13

Kruskal's Algorithm

QUESTION

Write a C++ program to find the MST of a given graph using Kruskal's algorithm.

AIM

To implement Kruskal's algorithm using the Disjoint Set (Union-Find) data structure.

THEORY

Kruskal's algorithm sorts edges from low to high weight and adds them to the spanning tree if they don't form a cycle. Cycle detection is managed by a Disjoint Set structure.

ALGORITHM

1. Sort all edges in non-decreasing order of weight.
2. Iterate through sorted edges. Check if vertices belong to different sets using Union-Find.
3. Unify sets and include the edge weight in the total if no cycle is formed.

COMPLEXITY ANALYSIS

Time: $O(E \log E)$

Space: $O(V + E)$

Output

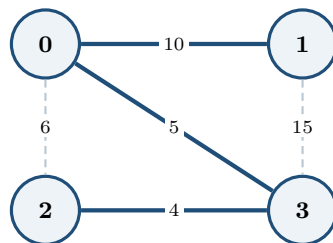
```

Enter the number of vertices: 4
Enter Edge (u v w) (Enter -1 -1 -1 To Stop Reading): 0 1 10
Enter Edge (u v w) (Enter -1 -1 -1 To Stop Reading): 0 2 6
Enter Edge (u v w) (Enter -1 -1 -1 To Stop Reading): 0 3 5
Enter Edge (u v w) (Enter -1 -1 -1 To Stop Reading): 1 3 15
Enter Edge (u v w) (Enter -1 -1 -1 To Stop Reading): 2 3 4
Enter Edge (u v w) (Enter -1 -1 -1 To Stop Reading): -1 -1 -1
Edges in the Minimum Spanning Tree:
2 - 3 : 4
0 - 3 : 5
0 - 1 : 10
Total Weight of MST: 19

```

Result

The five edges entered above form this graph. Thick edges are the ones Kruskal's algorithm accepts.



Solid = accepted, dashed = rejected because it would close a cycle.

Edges are first sorted by weight, giving (2-3:4) (0-3:5) (0-2:6) (0-1:10) (1-3:15), then taken in that order.

Edge	find(u)	find(v)	Decision	parent []
2-3 (4)	2	3	different sets $\Rightarrow$ accept , total 4	0,1,2,2
0-3 (5)	0	2	different sets $\Rightarrow$ accept , total 9	2,1,2,2
0-2 (6)	2	2	same set $\Rightarrow$ reject, cycle	2,1,2,2
0-1 (10)	2	1	different sets $\Rightarrow$ accept , total 19	2,2,2,2
1-3 (15)	2	2	same set $\Rightarrow$ reject, cycle	2,2,2,2

Three edges are accepted for four vertices, which is the expected $V - 1$, and the total weight is 19. The Disjoint Set is what makes the cycle test cheap: after the first two unions, vertices 0, 2 and 3 all have root 2, so `find(0)` and `find(2)` return the same value and edge 0-2 is rejected without inspecting the tree.

Program

```

1  #include <iostream>
2  #include <vector>
3  #include <algorithm>
4  using namespace std;
5  struct Edge {
6      int u, v, weight;
7      bool operator<(const Edge& other) const { return weight < other.weight; }
8  };
9  struct DisjointSet {
10     vector<int> parent, rank;
11     DisjointSet(int n) {
12         parent.resize(n); rank.resize(n, 0);
13         for (int i = 0; i < n; i++) parent[i] = i;
14     }
15     int find(int i) {
16         if (parent[i] == i) return i;
17         return parent[i] = find(parent[i]);
18     }
19     bool unite(int i, int j) {
20         int root_i = find(i), root_j = find(j);
21         if (root_i != root_j) {
22             if (rank[root_i] < rank[root_j]) parent[root_i] = root_j;
23             else if (rank[root_i] > rank[root_j]) parent[root_j] = root_i;
24             else { parent[root_j] = root_i; rank[root_i]++; }
25             return true;
26         }
27         return false;
28     }
29 };
30 int main() {
31     int size, u, v, w;
32     cout << "Enter the number of vertices: ";
33     cin >> size;
34     vector<Edge> edges;
35     do {
36         cout << "Enter Edge (u v w) (Enter -1 -1 -1 To Stop Reading): ";
37         cin >> u >> v >> w;
38         if (u == -1 && v == -1 && w == -1) break;
39         edges.push_back({u, v, w});
40     } while (true);
41     sort(edges.begin(), edges.end());
42     DisjointSet ds(size);
43     int mst_weight = 0;
44     cout << "Edges in the Minimum Spanning Tree:\n";
45     for (const auto& edge : edges) {
46         if (ds.unite(edge.u, edge.v)) {
47             cout << edge.u << " - " << edge.v << " : " << edge.weight << endl;
48             mst_weight += edge.weight;
49         }
50     }
51     cout << "Total Weight of MST: " << mst_weight << endl;
52     return 0;
53 }

```


Experiment 14

Dijkstra's Algorithm

QUESTION

Write a C++ program to compute the shortest paths from a source using Dijkstra's algorithm.

AIM

To find the shortest paths in a non-negative weighted graph using a Priority Queue.

THEORY

Dijkstra's algorithm tracks shortest distances from the source and continually relaxes the edges of the minimally distant node until all nodes are processed.

ALGORITHM

1. Initialize distances to infinity, except the source (0). Insert source into Min-Heap.
2. Extract the minimum distance vertex.
3. For adjacent vertices, if `current dist + weight < known dist`, update and push to Heap.

COMPLEXITY ANALYSIS

Time: $O(E \log V)$

Space: $O(V + E)$

Output

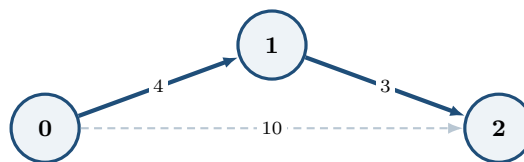
```

Enter the number of vertices: 3
Enter Neighbour Vertex Distance of Vertex 0 (-1,-1 To Stop Reading): 1 4
Enter Neighbour Vertex Distance of Vertex 0 (-1,-1 To Stop Reading): 2 10
Enter Neighbour Vertex Distance of Vertex 1 (-1,-1 To Stop Reading): -1 -1
Enter Neighbour Vertex Distance of Vertex 1 (-1,-1 To Stop Reading): 2 3
Enter Neighbour Vertex Distance of Vertex 1 (-1,-1 To Stop Reading): -1 -1
Enter Neighbour Vertex Distance of Vertex 2 (-1,-1 To Stop Reading): -1 -1
Distance from Node 0 to Node 0 is : 0
Distance from Node 0 to Node 1 is : 4
Distance from Node 0 to Node 2 is : 7

```

Result

The adjacency list gives this directed weighted graph. Thick arcs form the final shortest-path tree from source 0.



The direct arc $0 \rightarrow 2$ of weight 10 loses to the two-hop route of cost 7.

Step	Popped	Relaxations attempted	dist []
—	—	initial state	[0, INF, INF]
1	(d=0, v=0)	$0+4 = 4 < \text{INF}$ update ; $0+10 = 10 < \text{INF}$ update	[0, 4, 10]
2	(d=4, v=1)	$4+3 = 7 < 10$ update dist[2]	[0, 4, 7]
3	(d=7, v=2)	vertex 2 has no outgoing edges	[0, 4, 7]
4	(d=10, v=2)	stale entry: $10 > \text{dist}[2] = 7$, so continue	[0, 4, 7]

Step 4 shows why the `if (d > dist[u]) continue;` guard matters. Vertex 2 was pushed twice — once with 10 and later with 7 — and the heap still holds the obsolete entry. The guard discards it instead of relaxing vertex 2's edges a second time.

Program

```
1  #include<iostream>
2  #include<vector>
3  #include<queue>
4  #include<climits>
5  using namespace std;
6  vector<int> run_dijkstra(const vector<vector<pair<int,int>>>& adj, int start) {
7      vector<int> dist(adj.size(),INT_MAX);
8      dist[start] = 0;
9      priority_queue<pair<int,int>, vector<pair<int,int>>, greater<pair<int,int>>> pq;
10     pq.push({0,start});
11     while (!pq.empty()) {
12         auto [d,u] = pq.top(); pq.pop();
13         if (d>dist[u]) continue;
14         for (auto [v,w] : adj[u]) {
15             if (dist[u]+w < dist[v]) {
16                 dist[v] = dist[u]+w;
17                 pq.push({dist[v],v});
18             }
19         }
20     }
21     return dist;
22 }
23 int main() {
24     int v, d, size;
25     cout<<"Enter the number of vertices: ";
26     cin>>size;
27     vector<vector<pair<int,int>>> adj(size);
28     for (int i=0;i<size;i++) {
29         do {
30             cout<<"Enter Neighbour Vertex Distance of Vertex "<<i<<" (-1,-1 To Stop Reading): ";
31             cin>>v>>d;
32             if (v == -1 && d == -1) break;
33             adj[i].push_back({v,d});
34         } while (true);
35     }
36     vector<int> distances = run_dijkstra(adj,0);
37     for (int i=0;i<size;i++) {
38         cout<<"Distance from Node 0 to Node "<<i<<" is : "<<distances[i]<<endl;
39     }
40     return 0;
41 }
```


Experiment 15

Bellman-Ford Algorithm

QUESTION

Write a C++ program to find the shortest path utilizing the Bellman-Ford algorithm.

AIM

To implement Bellman-Ford, which inherently supports graphs with negative weight edges.

THEORY

The algorithm relaxes all edges $V - 1$ times. If distances change in a V^{th} pass, a negative weight cycle exists.

ALGORITHM

1. Initialize distance from source to infinity, and source to 0.
2. Repeat $V - 1$ times: for every edge (u, v, w) , if $\text{dist}[u] + w < \text{dist}[v]$, update $\text{dist}[v] = \text{dist}[u] + w$.
3. Make one further pass over all edges. If any distance can still be reduced, the graph contains a negative weight cycle.

COMPLEXITY ANALYSIS

Time: $O(V \cdot E)$

Space: $O(V)$

Output

```
Enter the number of vertices: 3
Enter Edge (u v w) (Enter -1 -1 -1 To Stop Reading): 0 1 4
Enter Edge (u v w) (Enter -1 -1 -1 To Stop Reading): 1 2 3
Enter Edge (u v w) (Enter -1 -1 -1 To Stop Reading): 0 2 10
Enter Edge (u v w) (Enter -1 -1 -1 To Stop Reading): -1 -1 -1
Distance from Node 0 to Node 0 is : 0
Distance from Node 0 to Node 1 is : 4
Distance from Node 0 to Node 2 is : 7
```

Result

Unlike Dijkstra's algorithm, Bellman-Ford keeps no priority queue: it simply sweeps the whole edge list $V - 1 = 2$ times, in the order the edges were entered.

Pass	Edge	Test	dist[]
—	—	initial state	[0, INF, INF]
1	$0 \rightarrow 1$ (4)	$0+4 = 4 < \text{INF}$ update	[0, 4, INF]
1	$1 \rightarrow 2$ (3)	$4+3 = 7 < \text{INF}$ update	[0, 4, 7]
1	$0 \rightarrow 2$ (10)	$0+10 = 10 \not< 7$, no change	[0, 4, 7]
2	$0 \rightarrow 1$ (4)	$0+4 = 4 \not< 4$, no change	[0, 4, 7]
2	$1 \rightarrow 2$ (3)	$4+3 = 7 \not< 7$, no change	[0, 4, 7]
2	$0 \rightarrow 2$ (10)	$0+10 = 10 \not< 7$, no change	[0, 4, 7]

Verification pass: no edge reduces any distance $\Rightarrow$ no negative weight cycle.

Pass 1 already settles every distance here because the edges happened to be entered in a convenient order. In the worst case a shortest path uses all $V - 1$ edges and one pass fixes only one more vertex, which is where the $O(V \cdot E)$ bound comes from. The verification pass does not shorten any path, so the program prints the distances rather than reporting a negative weight cycle.

Program

```
1  #include <iostream>
2  #include <vector>
3  #include <climits>
4  using namespace std;
5  struct Edge {
6      int u, v, weight;
7  };
8  bool run_bellman_ford(int size, const vector<Edge>& edges, int start, vector<int>& dist) {
9      dist.assign(size, INT_MAX);
10     dist[start] = 0;
11     for (int i = 1; i <= size - 1; i++) {
12         for (const auto& edge : edges) {
13             if (dist[edge.u] != INT_MAX && dist[edge.u] + edge.weight < dist[edge.v]) {
14                 dist[edge.v] = dist[edge.u] + edge.weight;
15             }
16         }
17     }
18     for (const auto& edge : edges) {
19         if (dist[edge.u] != INT_MAX && dist[edge.u] + edge.weight < dist[edge.v]) {
20             return false;
21         }
22     }
23     return true;
24 }
25 int main() {
26     int size, u, v, w;
27     cout << "Enter the number of vertices: ";
28     cin >> size;
29     vector<Edge> edges;
30     do {
31         cout << "Enter Edge (u v w) (Enter -1 -1 -1 To Stop Reading): ";
32         cin >> u >> v >> w;
33         if (u == -1 && v == -1 && w == -1) break;
34         edges.push_back({u, v, w});
35     } while (true);
36     vector<int> distances;
37     if (!run_bellman_ford(size, edges, 0, distances)) {
38         cout << "Graph contains a negative weight cycle" << endl;
39         return 0;
40     }
41     for (int i = 0; i < size; i++) {
42         cout << "Distance from Node 0 to Node " << i << " is : " << distances[i] << endl;
43     }
44     return 0;
45 }
```


Experiment 16

Floyd-Warshall Algorithm

QUESTION

Write a C++ program to find the all-pairs shortest paths using the Floyd-Warshall algorithm.

AIM

To find the shortest distances between every pair of vertices in a given graph via Dynamic Programming.

THEORY

Floyd-Warshall considers all vertices as intermediate nodes. It iteratively updates the distance between pairs (i, j) if passing through a node k yields a shorter path.

ALGORITHM

1. Initialize an $N \times N$ matrix. Use infinity for missing edges.
2. For intermediate k , iterate over all (i, j) pairs.
3. Update $\text{dist}[i][j]$ if $\text{dist}[i][k] + \text{dist}[k][j]$ is shorter.

COMPLEXITY ANALYSIS

Time: $O(V^3)$

Space: $O(V^2)$

Output

```
Enter the number of vertices: 3
Enter the adjacency matrix (type 'INF' for infinity):
0 3 INF
INF 0 1
INF INF 0
Shortest path matrix:
0 3 4
INF 0 1
INF INF 0
```

Result

The outer loop fixes an intermediate vertex k and asks, for every pair (i, j) , whether going $i \rightarrow k \rightarrow j$ beats the best route known so far. The one entry that ever changes is shown in bold.

Initial	After $k = 0$	After $k = 1$	After $k = 2$
$\begin{bmatrix} 0 & 3 & \infty \\ \infty & 0 & 1 \\ \infty & \infty & 0 \end{bmatrix}$	$\begin{bmatrix} 0 & 3 & \infty \\ \infty & 0 & 1 \\ \infty & \infty & 0 \end{bmatrix}$	$\begin{bmatrix} 0 & 3 & \mathbf{4} \\ \infty & 0 & 1 \\ \infty & \infty & 0 \end{bmatrix}$	$\begin{bmatrix} 0 & 3 & \mathbf{4} \\ \infty & 0 & 1 \\ \infty & \infty & 0 \end{bmatrix}$

k What changes

- 0 No pair improves. Reaching vertex 0 from 1 or 2 is impossible ($\text{dist}[i][0] = \infty$), so 0 is useless as a stepping stone.
- 1 $\text{dist}[0][2]$ improves: $\text{dist}[0][1] + \text{dist}[1][2] = 3 + 1 = 4 < \infty$.
- 2 No pair improves. Vertex 2 has no outgoing edges, so $\text{dist}[2][j] = \infty$ for $j \neq 2$.

Only one cell changes in the whole run, and the ∞ entries persist because the graph is directed: there is genuinely no path back from 1 or 2 to 0. The guard `dist[i][k] != INT_MAX && dist[k][j] != INT_MAX` is what keeps those ∞ values from being added together and overflowing.

Program

```
1  #include <iostream>
2  #include <vector>
3  #include <string>
4  #include <climits>
5  using namespace std;
6  void run_floyd_warshall(vector<vector<int>>& dist, int size) {
7      for (int k = 0; k < size; k++) {
8          for (int i = 0; i < size; i++) {
9              for (int j = 0; j < size; j++) {
10                 if (dist[i][k] != INT_MAX && dist[k][j] != INT_MAX && dist[i][k] + dist[k][j] < dist[i][j]) {
11                     dist[i][j] = dist[i][k] + dist[k][j];
12                 }
13             }
14         }
15     }
16 }
17 int main() {
18     int size;
19     cout << "Enter the number of vertices: ";
20     cin >> size;
21     vector<vector<int>> dist(size, vector<int>(size));
22     cout << "Enter the adjacency matrix (type 'INF' for infinity):\n";
23     for (int i = 0; i < size; i++) {
24         for (int j = 0; j < size; j++) {
25             string s;
26             cin >> s;
27             if (s == "INF" || s == "inf") dist[i][j] = INT_MAX;
28             else dist[i][j] = stoi(s);
29         }
30     }
31     run_floyd_warshall(dist, size);
32     cout << "Shortest path matrix:\n";
33     for (int i = 0; i < size; i++) {
34         for (int j = 0; j < size; j++) {
35             if (dist[i][j] == INT_MAX) cout << "INF ";
36             else cout << dist[i][j] << " ";
37         }
38         cout << endl;
39     }
40     return 0;
41 }
```